

**Tổng quan về KNN, K-Means, SVM,
Perceptron, RNN/LSTM và Genetic
Algorithm trong mối liên hệ với
Reinforcement Learning**

Mục lục

1	EXPLAIN SOME DEFINITIONS	3
1.1	Bức tranh tổng quát	3
1.2	KNN — K-Nearest Neighbors	3
1.2.1	KNN là gì?	3
1.2.2	Mục đích	4
1.2.3	Ứng dụng cụ thể	4
1.2.4	Vì sao KNN không cần thiết trước khi học RL?	4
1.2.5	KNN có thể xuất hiện trong RL không?	5
1.3	K-Means	5
1.3.1	K-Means là gì?	5
1.3.2	Quy trình trực quan	5
1.3.3	Mục đích	5
1.3.4	Ứng dụng cụ thể	6
1.3.5	Vì sao K-Means không cần thiết trước khi học RL?	6
1.3.6	K-Means có thể dùng trong RL không?	6
1.4	SVM — Support Vector Machine	7
1.4.1	SVM là gì?	7
1.4.2	Trực giác	7
1.4.3	Kernel trong SVM	7
1.4.4	Mục đích	7
1.4.5	Ứng dụng cụ thể	7
1.4.6	Vì sao SVM không cần thiết trước khi học RL?	8
1.4.7	SVM có thể xuất hiện trong hệ thống RL không?	8
1.5	Perceptron	9
1.5.1	Perceptron là gì?	9
1.5.2	Mục đích	9

1.5.3	Ứng dụng cụ thể	9
1.5.4	Vì sao không cần học lại Perceptron?	9
1.5.5	Perceptron liên quan thế nào đến RL?	10
1.6	RNN và LSTM	10
1.6.1	RNN là gì?	10
1.6.2	Ví dụ	10
1.6.3	Vấn đề của RNN thông thường	11
1.6.4	LSTM là gì?	11
1.6.5	Mục đích	11
1.6.6	Ứng dụng cụ thể	11
1.6.7	Vì sao RNN/LSTM chưa cần cho RL cơ bản?	12
1.6.8	Khi nào RNN/LSTM cần thiết trong RL?	12
1.7	Genetic Algorithm	13
1.7.1	Genetic Algorithm là gì?	13
1.7.2	Ví dụ đơn giản	13
1.7.3	Mục đích	13
1.7.4	Ứng dụng cụ thể	13
1.7.5	Vì sao Genetic Algorithm không cần thiết cho RL?	14
1.7.6	Genetic Algorithm có liên quan đến RL không?	14
1.7.7	Khi nào GA hữu ích hơn?	15
1.8	So sánh mức độ cần thiết đối với lộ trình RL	15
1.9	Nên ưu tiên học gì thay cho các nội dung trên?	16

Chương 1

EXPLAIN SOME DEFINITIONS

1.1 Bức tranh tổng quát

Các thuật toán được đề cập thuộc nhiều nhóm khác nhau:

Thuật toán	Nhóm chính	Mục đích
KNN	Supervised Learning	Phân loại hoặc dự đoán dựa trên các mẫu gần nhất.
K-Means	Unsupervised Learning	Chia dữ liệu thành các cụm.
SVM	Supervised Learning	Tìm ranh giới phân loại tốt giữa các nhóm.
Perceptron	Supervised Learning	Bộ phân loại tuyến tính đơn giản, tiền thân của neural network.
RNN/LSTM	Deep Learning	Xử lý dữ liệu có thứ tự và bộ nhớ theo thời gian.
Genetic Algorithm	Evolutionary Optimization	Tìm lời giải tốt bằng mô phỏng tiến hóa.

Các thuật toán trên không phải nền tảng bắt buộc của Reinforcement Learning vì RL tập trung vào một kiểu bài toán khác:

$$\text{Agent} \rightarrow \text{Action} \rightarrow \text{Environment} \rightarrow \text{Reward, State.} \quad (1.1)$$

Các kiến thức cốt lõi của RL là Markov Decision Process, policy, value function, Bellman equation, exploration, Monte Carlo, Temporal Difference, SARSA và Q-Learning.

1.2 KNN — K-Nearest Neighbors

1.2.1 KNN là gì?

[Click video YouTube này.](#)

KNN dự đoán một mẫu mới bằng cách tìm K mẫu gần nó nhất trong dữ liệu đã biết.

Ví dụ, ta có dữ liệu về chiều cao, cân nặng và nhóm cơ thể. Khi có một người mới, KNN tìm những người có đặc điểm gần nhất rồi dựa trên nhãn của họ để dự đoán.

Với bài toán phân loại:

$$\hat{y} = \text{mode} \left(y_i \text{ của } K \text{ hàng xóm gần nhất} \right). \quad (1.2)$$

Với bài toán hồi quy:

$$\hat{y} = \frac{1}{K} \sum_{i \in N_K(x)} y_i, \quad (1.3)$$

trong đó $N_K(x)$ là tập K điểm gần nhất với x .

1.2.2 Mục đích

KNN dùng để:

- Phân loại.
- Hồi quy.
- Tìm kiếm mẫu tương tự.
- Gợi ý dựa trên độ giống nhau.

1.2.3 Ứng dụng cụ thể

- Phân loại hoa Iris dựa trên kích thước cánh hoa.
- Nhận dạng chữ số viết tay.
- Dự đoán giá nhà dựa trên các căn nhà tương tự.
- Tìm sản phẩm hoặc người dùng có đặc điểm gần nhau.
- Phát hiện mẫu bất thường nếu nó cách xa các mẫu còn lại.

1.2.4 Vì sao KNN không cần thiết trước khi học RL?

KNN giải quyết bài toán:

Với một đầu vào x , hãy dự đoán nhãn hoặc giá trị y .

Trong khi RL giải quyết bài toán:

Trong trạng thái s , nên chọn hành động a nào để tối đa hóa tổng phần thưởng tương lai?

KNN không cung cấp các khái niệm cốt lõi như reward, policy, return, Bellman equation, exploration hay Temporal Difference. Vì vậy, học KNN không giúp trực tiếp cho việc hiểu Q-Learning.

1.2.5 KNN có thể xuất hiện trong RL không?

Có, nhưng chủ yếu ở mức nâng cao hoặc trong các thiết kế đặc biệt:

- Tìm các trạng thái tương tự đã gặp trước đây.
- Ước lượng value dựa trên các trạng thái gần nhau.
- Episodic memory.
- Chọn hành động dựa trên các tình huống đã lưu.

1.3 K-Means

1.3.1 K-Means là gì?

K-Means là thuật toán chia dữ liệu thành K cụm sao cho các điểm trong cùng cụm gần nhau.

Ví dụ, dữ liệu khách hàng có thể gồm tuổi, thu nhập và mức chi tiêu. K-Means có thể tự động chia khách hàng thành các nhóm có hành vi tương tự mà không cần nhãn trước.

K-Means tìm các tâm cụm $\mu_1, \mu_2, \dots, \mu_K$ để tối thiểu hóa:

$$J = \sum_{i=1}^n \|x_i - \mu_{c_i}\|^2, \quad (1.4)$$

trong đó:

- x_i là điểm dữ liệu.
- c_i là cụm của điểm i .
- μ_{c_i} là tâm của cụm đó.

1.3.2 Quy trình trực quan

1. Chọn ngẫu nhiên K tâm cụm.
2. Gán mỗi điểm vào tâm gần nhất.
3. Tính lại tâm của từng cụm.
4. Lặp lại đến khi các tâm gần như không đổi.

1.3.3 Mục đích

K-Means dùng để khám phá cấu trúc trong dữ liệu chưa có nhãn.

1.3.4 Ứng dụng cụ thể

- Phân khúc khách hàng.
- Gom nhóm tài liệu theo chủ đề.
- Nén màu ảnh.
- Gom nhóm sản phẩm.
- Phân cụm vị trí địa lý.
- Tạo các nhóm người dùng có hành vi tương tự.

Ví dụ, một ảnh có hàng triệu màu có thể được rút xuống còn $K = 16$ màu đại diện để giảm dung lượng.

1.3.5 Vì sao K-Means không cần thiết trước khi học RL?

K-Means không học cách hành động và không sử dụng reward. Nó chỉ nhóm các điểm dữ liệu dựa trên độ giống nhau.

Nó không trả lời các câu hỏi như:

- Agent nên làm gì?
- Hành động nào đem lại reward cao?
- Giá trị tương lai của một trạng thái là bao nhiêu?
- Cách cập nhật policy như thế nào?
- Cách cân bằng exploration và exploitation ra sao?

Do đó, K-Means không phải nền tảng của Q-Learning, DQN hay PPO.

1.3.6 K-Means có thể dùng trong RL không?

K-Means có thể được dùng như một công cụ phụ trợ:

- Gom nhiều trạng thái liên tục thành một số trạng thái rời rạc.
- Tạo state abstraction.
- Gom nhóm các trải nghiệm trong replay buffer.

- Phân cụm hành vi của agent.
- Khám phá các vùng khác nhau trong không gian trạng thái.

Ví dụ, robot có vị trí liên tục (x, y) . Ta có thể dùng K-Means để gom hàng nghìn vị trí thành 50 vùng, rồi học Q-table trên 50 vùng đó.

1.4 SVM — Support Vector Machine

1.4.1 SVM là gì?

SVM là thuật toán phân loại tìm một đường hoặc siêu phẳng tách các lớp với khoảng cách biên lớn nhất.

Trong không gian hai chiều, ranh giới có thể được viết:

$$w^T x + b = 0. \quad (1.5)$$

SVM muốn các điểm của hai lớp nằm cách ranh giới càng xa càng tốt. Các điểm gần ranh giới nhất được gọi là *support vectors*; chúng quyết định vị trí của ranh giới.

1.4.2 Trực giác

Giả sử có hai nhóm điểm đỏ và xanh. Có nhiều đường có thể tách chúng, nhưng SVM chọn đường có khoảng trống giữa hai nhóm lớn nhất. Khoảng trống lớn thường giúp mô hình khái quát tốt hơn trên dữ liệu mới.

1.4.3 Kernel trong SVM

Nếu dữ liệu không thể tách bằng đường thẳng, SVM có thể sử dụng kernel để mô hình hóa ranh giới phi tuyến.

Một số kernel thường gặp:

- Linear kernel.
- Polynomial kernel.
- RBF kernel.

1.4.4 Mục đích

SVM chủ yếu dùng cho:

- Phân loại.
- Hồi quy thông qua Support Vector Regression.
- Phát hiện bất thường với One-Class SVM.

1.4.5 Ứng dụng cụ thể

- Phân loại email spam.
- Phân loại văn bản.
- Nhận dạng chữ viết tay.
- Phân loại ảnh khi lượng dữ liệu không quá lớn.
- Chẩn đoán bệnh từ các đặc trưng số.
- Phát hiện giao dịch bất thường.

SVM thường có ích khi dữ liệu có nhiều đặc trưng nhưng số lượng mẫu không quá lớn.

1.4.6 Vì sao SVM không cần thiết trước khi học RL?

SVM học một ánh xạ:

$$x \rightarrow y. \quad (1.6)$$

Ví dụ:

$$\text{email} \rightarrow \text{spam hoặc không spam}. \quad (1.7)$$

Trong RL, agent phải học một quá trình quyết định theo thời gian:

$$s_t \rightarrow a_t \rightarrow r_{t+1}, s_{t+1}. \quad (1.8)$$

SVM không trực tiếp giải quyết chuỗi hành động, reward trễ, return dài hạn, Bellman equation hay policy optimization. Vì vậy, hiểu SVM không giúp nhiều cho việc bắt đầu Q-Learning.

1.4.7 SVM có thể xuất hiện trong hệ thống RL không?

Có, thường là một module phụ:

- Phân loại vật thể trước khi agent ra quyết định.
- Phát hiện trạng thái nguy hiểm.
- Phân loại hành vi.
- Xây dựng bộ lọc an toàn.
- Xử lý đặc trưng đầu vào cho robot.

Ví dụ, SVM có thể phân loại ảnh camera thành “đường trống” hoặc “có vật cản”, sau đó agent RL quyết định đi thẳng hay rẽ.

1.5 Perceptron

1.5.1 Perceptron là gì?

Perceptron là một bộ phân loại tuyến tính đơn giản.

Nó tính:

$$z = w^\top x + b. \quad (1.9)$$

Sau đó đưa ra dự đoán:

$$\hat{y} = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases} \quad (1.10)$$

Nếu dự đoán sai, trọng số được cập nhật:

$$w \leftarrow w + \eta(y - \hat{y})x, \quad (1.11)$$

trong đó η là learning rate.

1.5.2 Mục đích

Perceptron dùng để phân loại hai lớp có thể tách bằng đường thẳng.

1.5.3 Ứng dụng cụ thể

- Phân loại điểm thành hai nhóm tuyến tính.
- Minh họa cách một neuron nhân tạo hoạt động.
- Học các cổng logic đơn giản như AND và OR.
- Làm nền tảng lịch sử để hiểu neural network.

Một perceptron có thể học được cổng AND, nhưng một perceptron đơn không giải được XOR vì XOR không tuyến tính.

1.5.4 Vì sao không cần học lại Perceptron?

Nếu đã học ANN, forward propagation, backpropagation và gradient descent thì kiến thức của bạn đã vượt xa perceptron đơn.

Một neural network nhiều lớp có thể được xem như nhiều đơn vị neuron kết hợp với nhau, sử dụng activation function để xử lý các quan hệ phi tuyến.

Perceptron hữu ích để hiểu lịch sử và trực giác, nhưng không phải điều kiện riêng biệt để học RL.

1.5.5 Perceptron liên quan thế nào đến RL?

Trong Deep RL, policy hoặc value function có thể được xấp xỉ bằng neural network:

$$Q(s, a; \theta) \quad (1.12)$$

hoặc:

$$\pi(a \mid s; \theta). \quad (1.13)$$

Perceptron có thể xem là dạng đơn giản nhất của function approximator. Tuy nhiên, DQN hoặc PPO trong thực tế sử dụng mạng neural nhiều lớp chứ không dùng perceptron đơn.

1.6 RNN và LSTM

1.6.1 RNN là gì?

[Click video YouTube này.](#)

RNN, hay Recurrent Neural Network, là mạng neural dành cho dữ liệu có thứ tự.

Điểm khác biệt là RNN có hidden state đóng vai trò như bộ nhớ:

$$h_t = f(W_x x_t + W_h h_{t-1} + b). \quad (1.14)$$

Dầu ra tại thời điểm t phụ thuộc vào:

- Dữ liệu hiện tại x_t .
- Trạng thái bộ nhớ trước đó h_{t-1} .

Do đó, RNN không chỉ nhìn một mẫu riêng lẻ mà còn sử dụng thông tin từ các bước trước.

1.6.2 Ví dụ

Trong câu “Tôi đã ăn một quả táo vì nó rất ngon”, để hiểu từ “nó” chỉ “quả táo”, mô hình cần nhớ các từ xuất hiện trước đó.

Trong dữ liệu nhiệt độ:

$$x_1, x_2, x_3, \dots, x_t, \quad (1.15)$$

dự đoán nhiệt độ ngày mai có thể phụ thuộc vào nhiều ngày trước.

1.6.3 Vấn đề của RNN thông thường

RNN cơ bản có thể gặp:

- Vanishing gradient.
- Exploding gradient.
- Khó nhớ quan hệ dài hạn.

1.6.4 LSTM là gì?

LSTM là một dạng RNN được thiết kế để duy trì thông tin lâu hơn bằng các gate.

Các thành phần chính gồm:

- Forget gate: quên thông tin không cần thiết.
- Input gate: thêm thông tin mới.
- Output gate: quyết định thông tin nào được xuất ra.
- Cell state: luồng bộ nhớ dài hạn.

LSTM không loại bỏ hoàn toàn các vấn đề gradient, nhưng xử lý phụ thuộc dài hạn tốt hơn RNN cơ bản.

1.6.5 Mục đích

RNN/LSTM dùng khi dữ liệu có quan hệ theo thời gian hoặc theo thứ tự.

1.6.6 Ứng dụng cụ thể

- Dự báo chuỗi thời gian.
- Nhận dạng giọng nói.
- Phân tích cảm xúc.
- Sinh văn bản.
- Dịch máy.
- Phát hiện bất thường trong chuỗi cảm biến.
- Dự đoán nhu cầu hoặc doanh số.
- Phân tích tín hiệu y tế như ECG.

Hiện nay, trong nhiều bài toán ngôn ngữ, Transformer thường được dùng thay RNN/LSTM. Tuy nhiên, RNN và LSTM vẫn hữu ích trong chuỗi thời gian, hệ thống nhỏ và một số bài toán điều khiển.

1.6.7 Vì sao RNN/LSTM chưa cần cho RL cơ bản?

Trong các môi trường RL nhập môn như FrozenLake, Taxi, CartPole hoặc MountainCar, trạng thái hiện tại thường đã chứa đủ thông tin để agent quyết định. Đây gần với giả định Markov:

$$P(S_{t+1} | S_t, A_t). \quad (1.16)$$

Agent không nhất thiết phải nhớ toàn bộ lịch sử.

Q-Learning dạng bảng chỉ cần:

$$Q(s, a). \quad (1.17)$$

DQN cơ bản cũng chỉ cần đưa trạng thái hiện tại vào mạng:

$$s_t \rightarrow Q(s_t, a_1), Q(s_t, a_2), \dots \quad (1.18)$$

Do đó, chưa cần RNN hoặc LSTM.

1.6.8 Khi nào RNN/LSTM cần thiết trong RL?

RNN/LSTM trở nên hữu ích khi agent không quan sát được toàn bộ trạng thái thật của môi trường. Trường hợp này thường được mô hình hóa bằng POMDP.

Ví dụ:

- Robot chỉ nhìn được một phần căn phòng.
- Game có thông tin ẩn.
- Điều khiển dựa trên cảm biến nhiễu.

Khi đó có thể dùng recurrent policy:

$$h_t = \text{LSTM}(o_t, h_{t-1}), \quad (1.19)$$

và:

$$a_t \sim \pi(a | h_t). \quad (1.20)$$

Các mô hình như DRQN hoặc Recurrent PPO sử dụng ý tưởng này. Vì vậy, RNN/LSTM hữu ích ở giai đoạn sau nhưng không cần trước Q-Learning hoặc DQN cơ bản.

1.7 Genetic Algorithm

1.7.1 Genetic Algorithm là gì?

Genetic Algorithm, hay GA, là phương pháp tối ưu lấy cảm hứng từ tiến hóa tự nhiên.

Thay vì tối ưu một lời giải duy nhất bằng gradient, GA duy trì một quần thể lời giải. Mỗi lời giải được biểu diễn như một chromosome.

Quy trình thường gồm:

1. Khởi tạo quần thể.
2. Đánh giá fitness.
3. Chọn các cá thể tốt.
4. Crossover để tạo cá thể mới.
5. Mutation để tạo biến đổi.
6. Lặp lại qua nhiều thế hệ.

1.7.2 Ví dụ đơn giản

Cần tìm tham số x làm hàm $f(x)$ đạt giá trị lớn nhất. GA tạo nhiều giá trị x , giữ lại các giá trị cho $f(x)$ lớn, sau đó kết hợp và biến đổi chúng để tìm nghiệm tốt hơn.

1.7.3 Mục đích

GA dùng để tìm lời giải trong các không gian:

- Rất lớn.
- Rời rạc.
- Không khả vi.
- Có nhiều cực trị cục bộ.
- Khó áp dụng gradient descent.

1.7.4 Ứng dụng cụ thể

- Bài toán người bán hàng TSP.
- Lập lịch sản xuất.
- Xếp lịch nhân viên.
- Tối ưu tuyến đường.

- Thiết kế cánh máy bay hoặc cấu trúc kỹ thuật.
- Tối ưu tham số mô hình.
- Tìm kiến trúc neural network.
- Tối ưu chiến lược trong game.
- Phân bổ nguồn lực.

Trong bài toán TSP, mỗi chromosome có thể là một thứ tự đi qua các thành phố.

1.7.5 Vì sao Genetic Algorithm không cần thiết cho RL?

GA là một phương pháp tối ưu, nhưng không cung cấp các khái niệm nền tảng của RL như:

- MDP.
- State value.
- Action value.
- Bellman equation.
- TD error.
- On-policy và off-policy.
- Credit assignment theo thời gian.

RL thường học từ từng transition:

$$(s_t, a_t, r_{t+1}, s_{t+1}), \quad (1.21)$$

trong khi GA thường chỉ đánh giá một lời giải hoặc một policy bằng một fitness tổng thể:

$$\text{fitness}(\theta) = \text{tổng reward của policy có tham số } \theta. \quad (1.22)$$

1.7.6 Genetic Algorithm có liên quan đến RL không?

Có. GA có thể dùng để tối ưu policy mà không cần backpropagation.

Giả sử policy là mạng neural:

$$a = \pi(s; \theta). \quad (1.23)$$

GA có thể coi toàn bộ trọng số θ là chromosome. Mỗi cá thể là một mạng neural khác nhau.

Quy trình:

1. Tạo nhiều mạng neural ngẫu nhiên.
2. Cho từng mạng tương tác với môi trường.
3. Dùng tổng reward làm fitness.
4. Giữ các mạng tốt nhất.
5. Lai ghép và đột biến trọng số.
6. Tiếp tục qua nhiều thế hệ.

Cách này thường được gọi là neuroevolution hoặc evolutionary reinforcement learning.

1.7.7 Khi nào GA hữu ích hơn?

GA có thể hữu ích khi:

- Reward không khả vi.
- Môi trường mô phỏng được nhưng không có gradient.
- Không gian chiến lược phức tạp.
- Có thể chạy nhiều agent song song.
- Muốn tìm cả kiến trúc mạng lẫn trọng số.
- Gradient-based RL không ổn định.

Tuy nhiên, GA thường cần nhiều lần đánh giá môi trường và có thể tốn dữ liệu hơn các thuật toán như PPO hoặc DQN.

1.8 So sánh mức độ cần thiết đối với lộ trình RL

Kiến thức	Mức độ cần trước RL	Có thể dùng trong RL khi nào?
KNN	Không cần	Tìm trạng thái tương tự, non-parametric value approximation.
K-Means	Không cần	Phân cụm trạng thái, state abstraction.
SVM	Không cần	Phân loại đầu vào, safety classifier, perception module.
Perceptron	Không cần học riêng nếu đã biết ANN	Hiểu function approximator đơn giản.

Kiến thức	Mức độ cần trước RL	Có thể dùng trong RL khi nào?
RNN/LSTM	Chưa cần	POMDP, quan sát thiếu, cần nhớ lịch sử.
Genetic Algorithm	Không cần	Neuroevolution, policy search không dùng gradient.

1.9 Nên ưu tiên học gì thay cho các nội dung trên?

Để bắt đầu RL, thứ tự ưu tiên nên là:

$$\text{Xác suất và kỳ vọng} \rightarrow \text{MDP} \rightarrow \text{Return} \rightarrow V(s), Q(s, a), \quad (1.24)$$

$$\rightarrow \text{Bellman equation} \rightarrow \text{Dynamic Programming} \rightarrow \text{Monte Carlo}, \quad (1.25)$$

$$\rightarrow \text{TD} \rightarrow \text{SARSA} \rightarrow \text{Q-Learning} \rightarrow \text{DQN}. \quad (1.26)$$

Có thể hiểu ngắn gọn:

- **KNN, K-Means, SVM:** các thuật toán ML cổ điển giải quyết phân loại, hồi quy hoặc phân cụm; không dạy agent cách ra quyết định theo thời gian.
- **Perceptron:** nền móng của ANN; nếu đã học ANN thì không cần quay lại quá sâu.
- **RNN/LSTM:** cần khi agent phải nhớ lịch sử; chưa cần cho RL cơ bản.
- **Genetic Algorithm:** một cách tối ưu policy thay thế; không phải nền tảng của Bellman-based RL.

Do đó, có thể học sơ lược các thuật toán này để có cái nhìn tổng quan về AI/ML, nhưng không nên trì hoãn việc học MDP, Bellman equation và Q-Learning chỉ để hoàn thành chúng.